Introduction to Robotics: ROS 2 Lab. 1.1

Python Introduction

Professor: Pietro Falco
Tutor: Alessandro Adami, email: adamialess@dei.unipd.it

Department of Information Engineering (DEI)
Università degli Studi di Padova

February 22, 2026

1. Why Python?
2. Python Basics
3. Objects and Types
4. Containers
5. Flow controllers
6. Functions
7. Libraries
8. Object-Oriented Programming (OOP)
9. Python Appendix: Advanced Tips
10. Hands-on

Python is a high-level, general-purpose programming language that has become the standard for Robotics, AI, and Data Science.

- **High-Level**: It abstracts away complex details like memory management (garbage collection) and pointer arithmetic.
- **Readability**: Often described as "executable pseudocode," making it ideal for rapid prototyping in research.
- **Batteries Included**: Comes with a massive standard library and an ecosystem of specialized packages (PyPI).
- **Dynamic Typing**: Variable types are determined at runtime, allowing for flexible and fast development.

Python in ROS 2

In ROS 2, Python allows you to write code that focuses on **logic and integration** without worrying about the underlying middleware boilerplate.

Understanding how code runs is key to choosing between C++ and Python in robotics.

Compiled (e.g., C++)

- Code is converted to **machine code** once before execution.
- **Pros**: Extremely fast; errors found at compile-time.
- **Use Case**: Real-time motor control.

Interpreted (e.g., Python)

- Code is executed **line-by-line** by an interpreter.
- **Pros**: Faster development; no "wait for build" cycle.
- **Use Case**: Path planning, Vision.

Execution Workflow

```
# C++ (Compiled)                # Python (Interpreted)
g++ node.cpp -o node    ->      python3 node.py
./node
```

One of Python's greatest strengths for learners is the **REPL** (Read-Eval-Print Loop), which allows you to test logic instantly.

- **Immediate Feedback**: Type a command, see the result. Perfect for testing math or library syntax.
- **Introspection**: Use `help()` or `dir()` to see what an object can do.

Using the Python Shell

```
python3
>>> 15 * 10 / 2
75.0
>>> import numpy as np
>>> np.pi
3.141592653589793
>>> exit()
```

Visual Studio Code

For a better experience, we often use **VSCode** or **Jupyter Notebooks**, which add features like tab-completion and "magic" commands.

Python is the language of choice for modern engineering laboratories due to its balance of simplicity and power.

- **Ease of Learning**: Python offers a concise and readable code structure.
- **Consistency**: It provides a consistent environment for scientific computing and data analysis.
- **Transition from MATLAB**: The syntax, especially with NumPy, is very similar to MATLAB, making it intuitive for engineering students.
- **Python vs. C++ in ROS 2**:
  - **C++** is used for high-performance, real-time critical robotics nodes.
  - **Python** is used for rapid prototyping, high-level logic, and integrating Learning models.
  - **The Bridge**: Many libraries (like `OpenCV` or ROS 2's core) are written in C++ but exposed to Python via **Pybind11**, giving you "C++ speed" with "Python ease."

Python Modules

Module	Application Field
`NumPy`	Scientific computing and data analysis.
`Scikit-learn`	General purpose Machine Learning.
`OpenCV`	Computer Vision for robotics.
`Pytorch`	Deep Learning frameworks.
. . .	. . .

Syntactic Hierarchy:

- Python programs are structured through **indentation** ⇥! No brackets or 'end' strings are necessary. All statements with the same distance to the right belong to the same block of code.
- Use a colon (:) to initiate a new nested block.
- Maintain consistent spacing to define the block's scope and reduce it to end the block. Blank and comment lines are ignored.

Example: Loop and Conditionals (print even and odd numbers)

```
for joint in range(10):
    print('Joint: ' + str(joint), end=' ')
    if joint % 2 == 0:
        print('even', end=' ') # This is a comment
    else:
        print('odd', end=' ')   # This is another comment
```

Remark

Do not mix tabs and spaces! This can lead to unexpected `IndentationErrors`.

Python programs manipulate data objects, which are categorized as follows:

- **Scalar:** Objects that cannot be subdivided.
- **Non-scalar:** Objects with internal structure that can be accessed.

Scalar Object Table

Type	Internal Name	Description
Integer	`int`	Any integer
Floating Point	`float`	Real numbers (64 bit)
Complex	`complex`	Imaginary unit is `j`
Boolean	`bool`	`True` or `False`
String	`str`	Sequence of characters
Bytes	`bytes`	A sequence of unsigned 8-bit, for manipulating binary data
None	`NoneType`	Null equivalent

Type Verification Examples

```python
type(10)                    # ---> <class 'int'>
type(7.3)                   # ---> <class 'float'>
type(1 + 2j)                # ---> <class 'complex'>
a = True
type(a)                     # ---> <class 'bool'>
type('Machine Learning')    # ---> <class 'str'>
type(None)                  # ---> <class 'NoneType'>
```

Type casting allows you to explicitly convert an object from one data type to another. This is essential when processing sensor data or user input.

- **float(x)**: Converts an integer or string to a floating-point number.
- **int(x)**: Converts a number or string to an integer. **Note**: It truncates the decimal part (does not round).
- **str(x)**: Converts an object into a string representation (useful for logging).

Type Cast Examples

```
# 1. Basic Casting
print(float(5))       # Output: 5.0
print(int(3.9))       # Output: 3 (Truncation, not rounding!)
# 2. Verifying Types
print(type(float(5))) # Output: <class 'float'>
print(type(int(3.9))) # Output: <class 'int'>
# 3. Practical Case: String to Number
sensor_reading = "1.234"
numeric_value = float(sensor_reading) # Converts 'string' to 'float'.
```

Precedence

In robotics, many ROS 2 messages arrive as specific types. Knowing how to cast ensures your mathematical operations (like calculating velocity) don't raise a `TypeError`.

Python supports various operators to manipulate scalar objects.

Arithmetic

Op.	Action
+	Addition
-	Subtraction
*	Multiplication
/	Division
//	Floor Div.
%	Module
**	Power

Comparison

Op.	Action
==	Equal to
!=	Not equal
>	Greater than
<	Less than
>=	Gt/Equal
<=	Lt/Equal

Logic/Str

Op.	Action
and	Logical AND
or	Logical OR
not	Logical NOT
+=	Increment
-=	Decrement
*=	Multiply by
**=	Power by
/=	Divide by
==	Comparison

Operation Examples

```python
print(17 / 3)    # Standard division returns a float: 5.666...
print(17 // 3)   # Floor division discards fractional part: 5
print(17 % 3)    # Module returns the remainder: 2
print('ItR' * 3) # String repetition: 'ItRItRItR'
```

The `input()` function allows your program to pause and wait for the user to provide data via the keyboard.

- **Display**: It prints the prompt string provided in the quotes to the console.
- **Capture**: The program waits until the user types something and hits Enter.
- **Storage**: The captured value is assigned to a variable for later use.
- **Type Warning**: `input()` **always** returns a string (`str`), regardless of what the user types.

Using input() in Scripts

```python
# 1. Basic string input
text = input("Type your name... ")
print("Your name is", text)

# 2. Numerical input (requires casting)
# We must wrap input() in int() or float() to perform math
a = int(input("Type first number... "))
b = int(input("Type second number... "))

print(f"The sum is: {a + b}")
```

Robotics Context

While ROS 2 nodes usually communicate via topics, the `input()` function is extremely useful in "Teleop" scripts or setup scripts where you need to manually define a target coordinate or a robot ID at startup.

Strings are sequences of characters where each element has a specific position.

- **0-based Indexing:** The first character is always at position 0.
- **Negative Indexing:** Access elements from the end using negative numbers (-1 is the last).
- **Slicing:** Extract a portion using `[start:stop]`, where **start** is included and **stop** is excluded.

Example: String Operations

```python
word = 'Python'
print(word[0])          # ---> 'P' (First character)
print(word[-1])         # ---> 'n' (Last character)
print(word[2:5])        # ---> 'tho' (Indices 2, 3, 4)
print(word[:2])         # ---> 'Py' (From start to index 2 excluded)
print(word[4:])         # ---> 'on' (From index 4 to the end)
print(word[-2:])        # ---> 'on' From index -2 to the end)
# 0  1  2  3  4  5
# P  y  t  h  o  n
#-6 -5 -4 -3 -2 -1
```

Rule of Thumb

Any string s can be reconstructed using the formula: `s[:i] + s[i:]`.

- Python web page
- Python documentation

Reference Documentation

The Python Library Reference is the main source for basic concepts, standard objects, and their methods. Keep it handy for refreshing your memory on built-in functions.

A list represents an ordered, mutable collection of any type of Python object.

- **Flexibility**: You can mix and match any type of object in a list.
- **Modifiability**: Elements can be added or removed at will.
- **Nested Lists**: Lists can contain other lists as elements.

Defining and Indexing Lists

```python
# Creating lists
empty_list = []                          # Standard definition
num_list = [1, 2, 3, 4]                  # List of integers
string_list = ['I', 'will', 'pass', 'the', 'exam']  # List of strings

# Nested lists
a = ['a', 'b', 'c']
n = [1, 2, 3]
x = [a, n]                              # x is now [['a', 'b', 'c'], [1, 2, 3]]
print(x)                                # Returns [['a', 'b', 'c'], [1, 2, 3]]
print(x[0])                             # Returns the first list: ['a', 'b', 'c']
```

Lists support a variety of built-in methods for data manipulation and inspection.

Membership

Operator	Action
`in`	Check if present
`not in`	Check if absent

Basic Ops

Operator	Action
`+`	Concatenate two lists
`*`	Repeat a list
`len()`	Get length

Modification

Method	Action
`.append()`	Add to end
`.extend()`	Extend a list
`.pop()`	Remove last
`.remove()`	Remove obj
`.index()`	Find index of elem.
`.sort()`	Sort the list, if exist
`.reverse()`	Reverse the list

Practical Examples

```python
int_list = [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
int_list.append(9)      # [10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 9]
int_list.remove(9)      # [10, 8, 7, 6, 5, 4, 3, 2, 1, 9] (removes first 9, raises error if 9 is not present)
int_list.pop()          # [10, 8, 7, 6, 5, 4, 3, 2, 1]
print(int_list[::-1])   # [1, 2, 3, 4, 5, 6, 7, 8, 10] (reverse order)
print(5 in int_list)    # Returns True
```

Indexing Note

List indexing and slicing follow the exact same rules as string indexing.

Understanding how Python handles object references is crucial for avoiding bugs. An immutable type means that the variable contains a deep copy of the inner value, while a mutable type means that modifications to the variable lead to changes to the referenced object.

Immutable Types

- `int`, `float`, `str`.
- Assigning `y = x` creates a copy of the **value**.
- Changing `x` does **not** affect `y`.

Mutable Types

- `lists`.
- Assigning `y = x` means both "point" to the **same memory**.
- Changing `x` **will** change `y`.

Example: The Difference in Practice

```
# Immutable behavior (int)
x = 5; y = x
x = 9              # y is still 5

# Mutable behavior (list)
a = [1, 2, 3]; b = a
a.append(4)        # b is now [1, 2, 3, 4]!
```

Key Definition

Mutable means the variable holds a reference to the memory address of the object. **Immutable** means the variable holds a fixed copy of the data.

When dealing with mutable objects (like lists or dictionaries), simply using `x = y` does not create a new object; it creates a new reference to the same memory.

- **Shallow Cloning**: The container is copied, but the nested items inside are still shared with the original.
- **Deep Cloning**: The container and all items inside it are copied as entirely new objects.

Cloning Examples

```python
import copy
y = [[1, 2], [3, 4]]
# 1. Alias (Reference sharing)
x = y              # x and y point to the same list
# 2. Shallow Copy
# Top-level list is new, but inner lists are still shared
x = copy.copy(y)
# 3. Deep Copy
# Everything is duplicated; x is completely independent
x = copy.deepcopy(y)
```

Why it matters

If you use a shallow copy on a list of coordinates `[[x,y], [x,y]]` and modify one coordinate in the clone, it will change in the original too! Always use **deepcopy** if you need a truly independent backup of complex data.

Tuples are ordered collections that, unlike lists, cannot be changed after creation.

- **Definition:** Created using parentheses `()` or just comma-separated values .
- **Immutability:** Once a tuple is created, you cannot add, remove, or replace elements.
- **Operations:** Support indexing, slicing, and membership tests (`in`) just like lists .

Tuple Syntax and Usage

```
num_tuple = (1, 6)              # Standard definition
print(num_tuple[0])             # Indexing works: 1
print(len(num_tuple))           # Length: 2

# num_tuple[0] = 10             # This would raise a TypeError!
```

Why use Tuples?

If lists do more, why bother?

1. **Efficiency:** Tuples are lighter and more memory efficient.
2. **Safety:** They provide data integrity (constants).
3. **Functionality:** Tuples can be used as keys in a dictionary, whereas lists cannot.

A dictionary consists of a collection of key-value pairs where each key maps to a specific value.

- **Definition:** Created using `dict()` or curly braces {}.
- **Keys:** Must be unique and immutable (e.g., strings, numbers, or tuples).
- **Values:** Can be any Python object and can be duplicated.

Dictionary Operations

```python
name2grade = dict() # Create empty dictionary (or use {})
# Adding entries
name2grade['Fabio'] = 18
name2grade['Leonardo'] = 31
name2grade['Alessandro'] = 30
#or name2grade = dict(Fabio=18, Leonardo=31,...)
#or name2grade = {'Fabio': 18, 'Leonardo': 31, ...}
print(name2grade['Alessandro']) # Accessing a value: 30
# Common Methods
keys = name2grade.keys()      # Get all keys
vals = name2grade.values()    # Get all values
items = name2grade.items()    # Get all (key, value) tuples
```

Memory Efficiency

Dictionaries use a technique called hashing to allow nearly instantaneous data retrieval, regardless of the size of the dictionary.

A **set** is an unordered, mutable collection of unique elements. A **frozenset** is its immutable counterpart.

- **Uniqueness**: Sets automatically discard duplicate values.
- **Syntax**: Uses {} but without key-value pairs (no colons).
- **Empty Sets**: Use `set()`, as {} creates an empty dictionary.

Declarations and Operations

```
num_set = {1, 2, 3}    # Standard Set, empty_dict = {} is NOT an empty set
char_set = set(('a', 'b', 'c'))  # From tuple, empty_set = set()
odd_fset = frozenset([1,1,1,1])  # empty_fset = frozenset()
# Common Boolean Operations (Method vs Operator)
a.issubset(b)            # a <= b   (Is 'a' inside 'b'?)
a.union(b)               # a | b    (All elements from both)
a.intersection(b)        # a & b    (Only elements in both)
a.difference(b)          # a - b    (Elements in 'a' but not 'b')
a.add(5)                 # Add element (Sets only)
```

Collection Property Summary

Type	Ordered	Mutable
`list`	Yes	Yes
`tuple`	Yes	No
`set`	No	Yes
`fset`	No	No

Python uses `if`, `elif`, and `else` to branch code execution based on conditions.

- **Syntax**: Each condition must end with a colon (`:`).
- **Indentation**: The code block following the colon must be indented to define its scope.
- **Logic**: Conditions can be combined using boolean operators like `and`, `or`, and `not`.

Example: Grading Logic

```python
grade = 27.2
course = "Calculus"

if (grade > 26) and (not (course != "ItR")):
    print("Everything is good") # Executed if both are True
else:
    print("Need to study for ItR!") # Executed otherwise

# Simple branching
x = 1
if x > 2:
    print('x > 2')
elif x == 2:
    print('x = 2')
else:
    print('x < 2')
```

The `for` loop is used to iterate over a sequence (list, tuple, or string).

- **Direct Iteration**: Access elements directly from the collection.
- **Range-based**: Use `range(stop)` or `range(start, stop, step)` for index-based loops.
- **Enumerate**: Use `enumerate()` to get both the index and the value simultaneously.

Loop Variations

```python
num_list = [1, 10, 42]

# Method 1: Direct
for x in num_list:
    print(x)
# Method 2: range() with length
for i in range(len(num_list)):
    print(num_list[i])
# Method 3: enumerate() for index and value
for index, element in enumerate(num_list):
    print('Index: ' + str(index) + ', Value: ' + str(element))
```

The range() Function

Note that `range(start, stop, step)` follows the slicing rule: the `start` is included, but the `stop` is excluded.

The `for` statement can be used to traverse dictionary keys, values, or both simultaneously.

- **Keys**: Use `.keys()` (default behavior) to loop through all unique identifiers.
- **Values**: Use `.values()` to loop through the associated data.
- **Items**: Use `.items()` for "tuple unpacking" to get both key and value.

Dictionary Iteration Examples

```python
grades = {30: ['Leonardo', 'Alessandro'], 26: ['Fabio', 'Luca']}

# 1. Iterate over keys
for key in grades.keys():
    print("Grade: " + str(key))

# 2. Iterate over values
for students in grades.values():
    print("Students: " + str(students))

# 3. Iterate over both (Pythonic way)
for key, val in grades.items():
    print("Key: " + str(key) + " | Value: " + str(val))
```

The `while` loop repeats a block of code as long as a condition remains `True`.

- **Syntax**: Standard syntax with flow controllers similar to C and other languages.
- **Break**: Immediately terminates the loop.
- **Continue**: Skips the rest of the current iteration and moves to the next one.

While Loop and Flow Control

```python
counter = 1
while counter < 10:
    counter = counter + 1        # Loop stops once counter == 10

counter = 1
while counter < 10:
    counter = counter + 1
    if counter == 5:
        break        # Loop stops entirely when counter reaches 5

counter = 1
while counter < 10:
    counter = counter + 1
    if counter in [5, 6, 9]:
        continue     # Skips printing for 5, 6, and 9
    print(counter)
```

In Python, functions are flexible objects that can be stored in data structures or passed as arguments to other functions.

- **Return Values**: All functions return a value; if not specified, they return `None`.
- **Arguments**: Supports positional, keyword, and default arguments. When used as arguments of a function, mutable variables are passed by reference, which means that any change to the variable within the function will also be maintained outside the function!
- **Multiple Outputs**: Tuples are commonly used to return more than one value.

Defining and Calling Functions

```python
# b has a default value of 2
def my_add(a, b=2):
    return a + b
# Using conditional logic within functions
def my_div(a, b):
    if b == 0:
        return 'Division by zero.'
    else:
        return a / b
#The declaration happens once, the body happens every time it is called.
print(my_add(1, 6))       # Positional arguments: 7
print(my_add(5))          # Uses default: 5 + 2 = 7
print(my_div(b=4, a=12)) # Keyword arguments (order doesn't matter)
```

Modules allow us to exploit libraries written by expert programmers to improve our code's efficiency.

- **Definition**: A module is a file containing Python definitions and statements.
- **Standard Libraries**: For this course, we mainly use `numpy` and **ROS2** libraries
- **Syntax**: Use `import` with an alias (`as`) or import specific functions using `from`.

Importing Examples

```python
# 1. Import with an alias (standard practice)
import numpy as np
arr = np.zeros((3, 4)) # Must use the 'np' prefix
# 2. Import a specific function
from numpy import ones
arr = ones((3, 4))     # No prefix needed anymore
```

Key Tip

Using an alias like `np` is common practice to keep code concise while still identifying which library a function belongs to.

In Robotics, we often treat robots as **Objects**. A class is a blueprint that defines what a robot *has* (data) and what it *can do* (methods).

- `self`: A reference to the current instance of the class. It allows access to variables across different functions.
- `__init__`: The constructor. It runs automatically when you create a new object.

Defining a Robot Class

```python
class Robot:
    def __init__(self, name, type="Mobile"):
        self.name = name            # Attribute
        self.type = type
        self.battery = 100          # Internal state

    def report(self):
        print(f"[{self.name}] {self.type} Robot - Battery: {self.battery}%")

# Create instances (Objects)
r1 = Robot("Turtle1", "Differential")
r2 = Robot("Drone1", "Quadrotor")
r1.report()
```

In ROS 2, nodes are persistent. They stay alive waiting for events. For a function (callback) to remember data from the past, we use **attributes** via `self`.

- **Local Variables**: (e.g., `x = 10`) Exist only while the function is running.
- **Attributes**: (e.g., `self.x = 10`) Belong to the Object. They stay in memory as long as the robot/node exists.

The Importance of self

```python
class SimpleCounter(Node):
    def __init__(self):
        super().__init__('counter_node')
        self.count = 0  # This 'count' will persist

    def timer_callback(self):
        # We access the PERSISTENT variable using self
        self.count += 1
        print(f"Counter is now: {self.count}")
```

Rule of Thumb

If you need a variable to be "remembered" between two different functions (e.g., from `init` to `callback`), you **must** prefix it with `self`.

In ROS 2, we don't write nodes from scratch. We **inherit** from the base `Node` class provided by the `rclpy` library.

- **Inheritance**: A way to derive a new class from an existing one, taking all its features.
- `super().__init__()`: Calls the parent class constructor (initializes the ROS 2 plumbing).

A Conceptual ROS 2 Node

```python
class MyFirstNode(Node): # Inheriting from ROS 2 Node class
    def __init__(self):
        # Initialize the parent Node with the name 'my_node'
        super().__init__('my_node')

        self.counter = 0
        print("ROS 2 Node has started!")

    def timer_callback(self):
        self.counter += 1
        print(f"Loop count: {self.counter}")
```

Why OOP for ROS 2?

Using classes allows the **Executor** to manage multiple nodes simultaneously and ensures that callbacks have access to the node's internal memory via `self`.

Inheritance allows a new class (Child) to take on the attributes and methods of an existing class (Parent). This promotes code reuse.

- **Base Class**: The original class containing general properties.
- **Derived Class**: The new class that "inherits" from the base.
- **Syntax**: `class Child(Parent):`

Basic Inheritance

```python
class Robot:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print(f"I am {self.name}, a generic robot.")

# Drone inherits from Robot
class Drone(Robot):
    def fly(self):
        print(f"{self.name} is taking off!")

my_drone = Drone("SkyHawk")
my_drone.greet()  # Inherited from Robot
my_drone.fly()    # Defined in Drone
```

A child class can provide a specific implementation of a method that is already defined in its parent class.

- **Customization**: The child "hides" the parent's version of the function.
- **Same Name**: The method name and arguments remain the same, but the logic inside changes.

Overriding Example

```python
class GenericSensor:
    def get_data(self):
        return "Generic raw data."

class Lidar(GenericSensor):
    # Overriding the parent's method
    def get_data(self):
        return [0.5, 1.2, 3.0, 0.8] # Returns a list of ranges instead

s1 = GenericSensor()
s2 = Lidar()
print(s1.get_data()) # "Generic raw data"
print(s2.get_data()) # [0.5, 1.2, 3.0, 0.8]
```

Instead of completely replacing a parent method, you can use `super()` to execute the parent's logic and then add your own.

- **Initialization**: Crucial for ROS 2 nodes to initialize the base `Node` class.
- **Efficiency**: Avoids rewriting the code already present in the parent.

Using super() in ROS 2 Style

```python
class MobileBase:
    def __init__(self, model):
        self.model = model
        print(f"Base {self.model} initialized.")

class DiffDriveRobot(MobileBase):
    def __init__(self, model, wheels):
        # Call the Parent's __init__
        super().__init__(model)
        self.wheels = wheels
        print(f"Differential drive with {self.wheels} wheels ready.")

my_bot = DiffDriveRobot("Turtle", 2)
```

NumPy is an optimized array-processing package with a syntax similar to MATLAB.

- **ndarray**: The main object, containing elements of the same type.
- **Dimensions**: Referred to as **axes**; the number of axes is called the **rank**.
- **Efficiency**: Highly optimized for matrix and vector operations.

Creating NumPy Arrays

```python
import numpy as np

a = np.array([1, 2, 3])                    # Single row vector
b = np.array([[1, 2, 3], [4, 5, 6]])       # 2x3 Matrix
c = np.eye(4)                              # 4x4 Identity matrix
d = np.random.random((2, 4))               # 2x4 Random matrix
```

MATLAB Users

If you are coming from MATLAB, you will find the NumPy syntax very intuitive for linear algebra tasks.

NumPy extends standard Python slicing to multiple dimensions using a comma-separated syntax.

Array Properties

Property	Code	Description
Dimensions	`d.ndim`	Number of axes (rank).
Shape	`d.shape`	Tuple of array dimensions.
Size	`d.size`	Total number of elements.

NumPy Slicing Examples

```python
d = np.random.random((2, 4))

print(d[:, :])        # The whole ndarray
print(d[1, 1])        # Element in 2nd row, 2nd column
print(d[:, 0:2])      # All rows, first 2 columns
print(d[0, 1:])       # 1st row, from 2nd element to the end
print(d[:, 1:-1])     # All rows, from 2nd to 3rd column
```

Memory Efficiency

Slicing a NumPy array creates a **view**, not a deep copy. Modifying the slice will modify the original array!

In Robotics, we constantly multiply matrices (e.g., $T_1^0 T_2^1$).

- **Element-wise (*)**: Multiplies $a_{ij} \times b_{ij}$. Rarely used for transformations.
- **Matrix Product (@)**: Standard row-by-column multiplication.

Matrix vs Element-wise

```python
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.eye(2)

print(A * B)  # Result: [[1, 0], [0, 4]] (Element-wise)
print(A @ B)  # Result: [[1, 2], [3, 4]] (Correct Matrix Product)
```

Rule of Thumb

Always use the **@** operator for composing transformations or multiplying a Jacobian by a joint velocity vector ($\dot{x} = J\dot{q}$).

In ROS 2 logging (`get_logger().info`), we use **f-strings** for readability and performance.

- **Syntax**: Prefix the string with an `f` and wrap variables in curly braces {}.
- **Auto-Conversion**: No need to manually call `str()` on numbers.
- **Precision Control**: Easily format decimals (crucial for coordinates).

f-string Examples

```python
robot_name = "TurtleBot3"
battery = 92.5678
pos_x, pos_y = 1.2, 3.4

# Basic usage
print(f"Status: {robot_name} is active.")

# Formatting decimals to 2 places
print(f"Robot at ({pos_x}, {pos_y}) - Battery: {battery:.2f}%")

# Evaluating expressions inside strings
print(f"Next ID: {100 + 1}")
```

To make a Python file executable as a standalone ROS 2 node, we use a specific entry point pattern.

- **Scope**: This block only executes if the script is run directly (e.g., `python3 my_node.py`).
- **Safety**: It prevents code from running automatically if the file is `imported` by another script.

The Standard Script Structure

```python
def main():
    print("Initializing ROS 2 Node...")
    # Your robotics logic goes here

if __name__ == '__main__':
    main()
```

ROS 2 Integration

The ROS 2 build system (`colcon`) uses this entry point to link your script to the `ros2 run` command.

Robotics often involves filtering large arrays (like Lidar scans). **List Comprehensions** provide a concise way to do this.

Filtering Sensor Data

```python
# A list of distances from a Lidar scan
lidar_ranges = [0.2, 1.5, 5.0, 0.8, 3.2, 0.4]

# Goal: Create a list of 'danger' obstacles closer than 1.0m
# The "Traditional" way:
danger_zone = []
for r in lidar_ranges:
    if r < 1.0:
        danger_zone.append(r)

# The "Pythonic" way (List Comprehension):
danger_zone = [r for r in lidar_ranges if r < 1.0]

print(f"Detected {len(danger_zone)} obstacles in the danger zone.")
```

In a robotics loop, a single error shouldn't crash the entire system. Use `try...except` to handle unexpected issues gracefully.

Handling Runtime Errors

```python
def calculate_velocity(distance, time):
    try:
        linear_vel = distance / time
        return linear_vel
    except ZeroDivisionError:
        print("Warning: Time is zero, robot is stationary.")
        return 0.0
    except TypeError:
        print("Error: Input must be numerical.")
        return None

print(calculate_velocity(10, 0)) # Handles error instead of crashing
```

When running a continuous control loop, you stop the script using Ctrl + C.
By default, Python shows a messy error. Use a `try-except` block for a clean exit.

Clean Loop Exit

```
try:
    while True:
        # read_sensors()
        # compute_control()
        print("Robot is running... (Ctrl+C to stop)")
except KeyboardInterrupt:
    print("\nStopping the robot safely...")
    # set_velocities_to_zero()
```

Inside the attached VS Code window, create a file named `test_env.py` and try the following:

test_env.py

```python
import numpy as np

def check_robot_status(dist_list):
    # Use list comprehension to filter distances < 1.0m
    obstacles = [d for d in dist_list if d < 1.0]

    for i, dist in enumerate(obstacles):
        # Use f-string for logging
        print(f"Warning: Obstacle {i} detected at {dist:.2f}m!")

if __name__ == "__main__":
    # Create a random array of 5 'laser' distances
    data = np.random.uniform(0.5, 3.0, 5)
    check_robot_status(data)
```

Run the test:

```
python3 test_env.py
```

Scenario: You are receiving a list of 10 distance readings from a Lidar sensor. Some readings are "noise" (negative values or zero), and some indicate that an obstacle is too close.
Goal: Write a script that:

1. Generates a list of 10 (or more) random floats between -1.0 and 5.0.
2. Uses a **list comprehension** to create a new list containing only valid distances (greater than 0).
3. Uses a **loop** to check if any valid distance is < 1.0m. If so, print: `"STOP: Obstacle at [distance]m"`.

lidar_filter.py

```python
import random
# Hint: use [random.uniform(-1, 5) for _ in range(10)]
# Your code here...
```

Scenario: A differential drive robot starts at $(0, 0)$ with orientation $\theta = 0$. It moves with a constant linear velocity v and angular velocity ω for a time dt.
Goal: Create a class `RobotPose` that updates the position using the following simplified equations:

$$x_{new} = x + v \cdot \cos(\theta) \cdot dt$$

$$y_{new} = y + v \cdot \sin(\theta) \cdot dt$$

$$\theta_{new} = \theta + \omega \cdot dt$$

odometry.py

```python
import numpy as np

class RobotPose:
    def __init__(self):
        self.x, self.y, self.theta = 0.0, 0.0, 0.0

    def update(self, v: float, omega: float, dt: float) -> None:
        # Update self.x, self.y, self.theta here
        pass

# Test: Move in a circle (v=1.0, omega=0.5) for 10 steps
```

Scenario: Practice data structures by creating a "Waypoints" system.

1. Create a dictionary where **keys** are strings (location names like "Home", "Lab", "Charging Dock") and **values** are tuples (x, y).
2. Write a function `get_distance(p1, p2)` that calculates the Euclidean distance between two points.
3. Iterate through the dictionary and print the distance from "Home" to all other waypoints.

waypoint_map.py

```
# Points: Home (0,0), Lab (5,2), Dock (1,4)
# Formula: dist = sqrt( (x2-x1)^2 + (y2-y1)^2 )
```